CSA4704-DEEP LEARNING

NAME:HARITHI S **REG NO:192421151**

Experiment 27: Deep Belief Network Representation Learning

Problem Statement: A postal service has very few labelled handwritten PIN-code digits but many unlabelled ones. It wants to pre-train features on the unlabelled data first. MNIST digits, using only 1,000 labels for fine-tuning. The objective is to build a DBN by stacking RBMs, pre-train greedily and fine-tune for classification. Use NumPy, SciPy, TensorFlow, Keras, Matplotlib.

Tasks:

1. Train two or three stacked RBMs layer by layer (784–500–250).

2. Visualise the learned weights of the first layer as image filters.

3. Transfer the weights into a Keras dense network and fine-tune with the labelled data.

4. Train the same network from random initialisation for comparison.

5. Compare accuracy and plot learned representations in 2-D.

FORMULA:

1. RBM Energy Function

$E(v, h) = -a^T v - b^T h - v^T W h$

2. Hidden Layer Activation

$h = \sigma(W^T v + b)$

3. Visible Layer Reconstruction

$\hat{v} = \sigma(Wh + a)$

4. Sigmoid Activation Function

$\sigma(x) = 1 / (1 + e^{-x})$

5. First RBM Weight Update

$W_1(new) = W_1(old) + \eta \times (v_0 h_0^T - v\square h\square^T)$

6. Second RBM Hidden Representation

$h_2 = \sigma(W_2^T h_1 + b_2)$

7. Greedy Layer-Wise Pre-training

$784 \rightarrow 500 \rightarrow 250$

$h_1 = \sigma(W_1^T v + b_1)$

$h_2 = \sigma(W_2^T h_1 + b_2)$

8. Output Layer Softmax Function

$P(y = j \mid x) = e^{zj} / \Sigma\square\ e^{zk}$

9. Cross-Entropy Loss

$L = -\Sigma_i\ y_i \log(\hat{y}_i)$

10. Classification Accuracy

Accuracy = (Correct Predictions / Total Predictions) × 100

11. Mean Squared Reconstruction Error

$MSE = (1/N) \times \Sigma_i\ (v_i - \hat{v}_i)^2$

12. Fine-Tuning Weight Update

$$W(new) = W(old) - \eta \times \partial L/\partial W$$

13. Training Accuracy

Training Accuracy = (Correct Training Predictions / Total Training Samples) × 100

14. Test Accuracy

Test Accuracy = (Correct Test Predictions / Total Test Samples) × 100

15. Two-Dimensional Representation

$Z = [h_1, h_2]$

where Z represents the learned features projected into two dimensions for visualisation.

CODE:

```
import numpy as np

import matplotlib.pyplot as plt

import tensorflow as tf

from tensorflow import keras

from scipy.special import expit

np.random.seed(42)

tf.random.set_seed(42)

(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()

x_train = x_train.reshape(-1, 784).astype("float32") / 255

x_test = x_test.reshape(-1, 784).astype("float32") / 255

x_unlabelled = (x_train[:10000] > 0.5).astype("float32")
```

```
x_labelled = x_train[:1000]

y_labelled = y_train[:1000]

learning_rate = 0.01

batch_size = 128

rbm_epochs = 5

def train_rbm(data, n_visible, n_hidden):
    W = np.random.normal(0, 0.01, (n_visible, n_hidden)).astype("float32")
    a = np.zeros(n_visible, dtype="float32")
    b = np.zeros(n_hidden, dtype="float32")
    for epoch in range(rbm_epochs):
        np.random.shuffle(data)
        for i in range(0, len(data), batch_size):
            v0 = data[i:i + batch_size]
            ph0 = expit(b + v0 @ W)
            h0 = (np.random.rand(*ph0.shape) < ph0).astype("float32")
            pv1 = expit(a + h0 @ W.T)
            v1 = (np.random.rand(*pv1.shape) < pv1).astype("float32")
            ph1 = expit(b + v1 @ W)
            W += learning_rate * (v0.T @ ph0 - v1.T @ ph1) / len(v0)
            a += learning_rate * np.mean(v0 - v1, axis=0)
```

```
            b += learning_rate * np.mean(ph0 - ph1, axis=0)

        print("RBM Epoch:", epoch + 1)

    return W, b

print("Training First RBM")

W1, b1 = train_rbm(x_unlabelled, 784, 500)

hidden1 = expit(b1 + x_unlabelled @ W1)

hidden1 = (np.random.rand(*hidden1.shape) < hidden1).astype("float32")

print("Training Second RBM")

W2, b2 = train_rbm(hidden1, 500, 250)


plt.figure(figsize=(10, 10))

for i in range(100):

    plt.subplot(10, 10, i + 1)

    plt.imshow(W1[:, i].reshape(28, 28), cmap="gray")

    plt.axis("off")

plt.suptitle("First RBM Learned Features")

plt.tight_layout()

plt.show()


def create_model(pretrained=False):
```

```
    model = keras.Sequential([
        keras.layers.Input(shape=(784,)),
        keras.layers.Dense(500, activation="sigmoid"),
        keras.layers.Dense(250, activation="sigmoid"),
        keras.layers.Dense(10, activation="softmax")
    ])
    if pretrained:
        model.layers[0].set_weights([W1, b1])
        model.layers[1].set_weights([W2, b2])
                        model.compile(optimizer="adam",       loss="sparse_categorical_crossentropy",
metrics=["accuracy"])
    return model


print("Training Pretrained DBN")
dbn_model = create_model(True)
dbn_model.fit(x_labelled, y_labelled, epochs=10, batch_size=32, validation_split=0.1)
print("Training Random Network")
random_model = create_model(False)
random_model.fit(x_labelled, y_labelled, epochs=10, batch_size=32, validation_split=0.1)
_, dbn_accuracy = dbn_model.evaluate(x_test, y_test, verbose=0)
```

```
_, random_accuracy = random_model.evaluate(x_test, y_test, verbose=0)

print("Pretrained DBN Accuracy:", round(dbn_accuracy * 100, 2), "%")

print("Random Network Accuracy:", round(random_accuracy * 100, 2), "%")

plt.figure(figsize=(7, 5))

plt.bar(["Pretrained DBN", "Random Network"], [dbn_accuracy * 100, random_accuracy * 100])

plt.title("Classification Accuracy Comparison")

plt.ylabel("Accuracy (%)")

plt.show()

dbn_features = keras.Model(dbn_model.input, dbn_model.layers[1].output).predict(x_test[:1000], verbose=0)

random_features = keras.Model(random_model.input, random_model.layers[1].output).predict(x_test[:1000], verbose=0)


def pca_2d(features):

    features = features - np.mean(features, axis=0)

    U, S, V = np.linalg.svd(features, full_matrices=False)

    return features @ V[:2].T


dbn_2d = pca_2d(dbn_features)

random_2d = pca_2d(random_features)

plt.figure(figsize=(8, 6))
```

```
plt.scatter(dbn_2d[:, 0], dbn_2d[:, 1], c=y_test[:1000], cmap="tab10", s=10)

plt.title("DBN Learned Representation")

plt.xlabel("Principal Component 1")

plt.ylabel("Principal Component 2")

plt.colorbar()

plt.show()

plt.figure(figsize=(8, 6))

plt.scatter(random_2d[:, 0], random_2d[:, 1], c=y_test[:1000], cmap="tab10", s=10)

plt.title("Random Network Representation")

plt.xlabel("Principal Component 1")

plt.ylabel("Principal Component 2")

plt.colorbar()

plt.show()
```

OUTPUT:

```
Training First RBM

RBM Epoch: 1

RBM Epoch: 2

RBM Epoch: 3

RBM Epoch: 4

RBM Epoch: 5
```

Training Second RBM

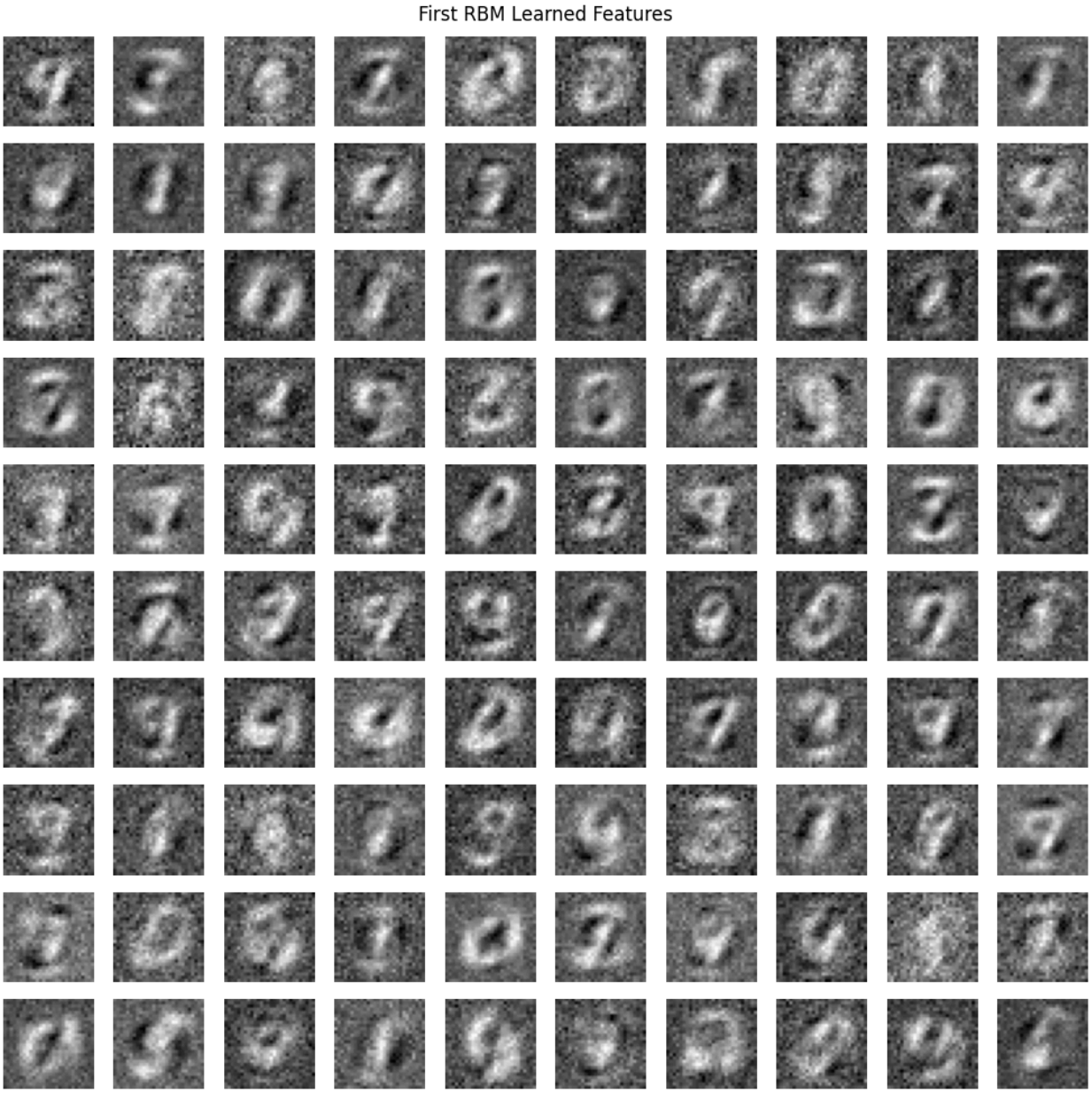
RBM Epoch: 1

RBM Epoch: 2

RBM Epoch: 3

RBM Epoch: 4

RBM Epoch: 5



Training Pretrained DBN

Epoch 1/10

29/29 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **3s** 53ms/step - accuracy: 0.3689 - loss: 1.9543 - val_accuracy: 0.4600 - val_loss: 1.6213

Epoch 2/10

29/29 ━━━━━━━━━━━━━━━━━━━━ **0s** 5ms/step - accuracy: 0.6178 - loss: 1.2822 - val_accuracy: 0.6400 - val_loss: 1.1456

Epoch 3/10

29/29 ━━━━━━━━━━━━━━━━━━━━ **0s** 6ms/step - accuracy: 0.7400 - loss: 0.8945 - val_accuracy: 0.7300 - val_loss: 0.8859

Epoch 4/10

29/29 ━━━━━━━━━━━━━━━━━━━━ **0s** 5ms/step - accuracy: 0.8233 - loss: 0.6763 - val_accuracy: 0.8000 - val_loss: 0.7230

Epoch 5/10

29/29 ━━━━━━━━━━━━━━━━━━━━ **0s** 5ms/step - accuracy: 0.8622 - loss: 0.5291 - val_accuracy: 0.8300 - val_loss: 0.6223

Epoch 6/10

29/29 ━━━━━━━━━━━━━━━━━━━━ **0s** 6ms/step - accuracy: 0.8911 - loss: 0.4273 - val_accuracy: 0.8500 - val_loss: 0.5665

Epoch 7/10

29/29 ━━━━━━━━━━━━━━━━━━━━ **0s** 5ms/step - accuracy: 0.9056 - loss: 0.3555 - val_accuracy: 0.8800 - val_loss: 0.5315

Epoch 8/10

29/29 ━━━━━━━━━━━━━━━━━━━━ **0s** 5ms/step - accuracy: 0.9200 - loss: 0.3005 - val_accuracy: 0.8800 - val_loss: 0.5121

Epoch 9/10

29/29 ━━━━━━━━━━━━━━━━━━━━ **0s** 5ms/step - accuracy: 0.9322 - loss: 0.2562 - val_accuracy: 0.8900 - val_loss: 0.5049

Epoch 10/10

29/29 ━━━━━━━━━━━━━━━━━━━━ **0s** 5ms/step - accuracy: 0.9456 - loss: 0.2205 - val_accuracy: 0.8900 - val_loss: 0.5067

Training Random Network

Epoch 1/10

29/29 ━━━━━━━━━━━━━━━━━━━━ **3s** 52ms/step - accuracy: 0.2933 - loss: 2.1303 - val_accuracy: 0.5000 - val_loss: 1.7910

Epoch 2/10

29/29 ━━━━━━━━━━━━━━━━━━━━ **0s** 5ms/step - accuracy: 0.6956 - loss: 1.3873 - val_accuracy: 0.7100 - val_loss: 1.1495

Epoch 3/10

29/29 ━━━━━━━━━━━━━━━━━━━━ **0s** 6ms/step - accuracy: 0.8022 - loss: 0.8820 - val_accuracy: 0.7500 - val_loss: 0.8331

Epoch 4/10

29/29 ━━━━━━━━━━━━━━━━━━━━ **0s** 5ms/step - accuracy: 0.8578 - loss: 0.6341 - val_accuracy: 0.8200 - val_loss: 0.6728

Epoch 5/10

29/29 ━━━━━━━━━━━━━━━━━━━━ **0s** 5ms/step - accuracy: 0.8844 - loss: 0.4952 - val_accuracy: 0.8600 - val_loss: 0.5859

Epoch 6/10

29/29 ━━━━━━━━━━━━━━━━━━━━ **0s** 5ms/step - accuracy: 0.9067 - loss: 0.4068 - val_accuracy: 0.8800 - val_loss: 0.5382

Epoch 7/10

29/29 ━━━━━━━━━━━━━━━━━━━━ **0s** 5ms/step - accuracy: 0.9189 - loss: 0.3456 - val_accuracy: 0.8800 - val_loss: 0.5117

Epoch 8/10

29/29 ━━━━━━━━━━━━━━━━━━━━ **0s** 6ms/step - accuracy: 0.9333 - loss: 0.3000 - val_accuracy: 0.8700 - val_loss: 0.4984

Epoch 9/10

29/29 ━━━━━━━━━━━━━━━━━━━━ **0s** 6ms/step - accuracy: 0.9378 - loss: 0.2630 - val_accuracy: 0.8800 - val_loss: 0.4967

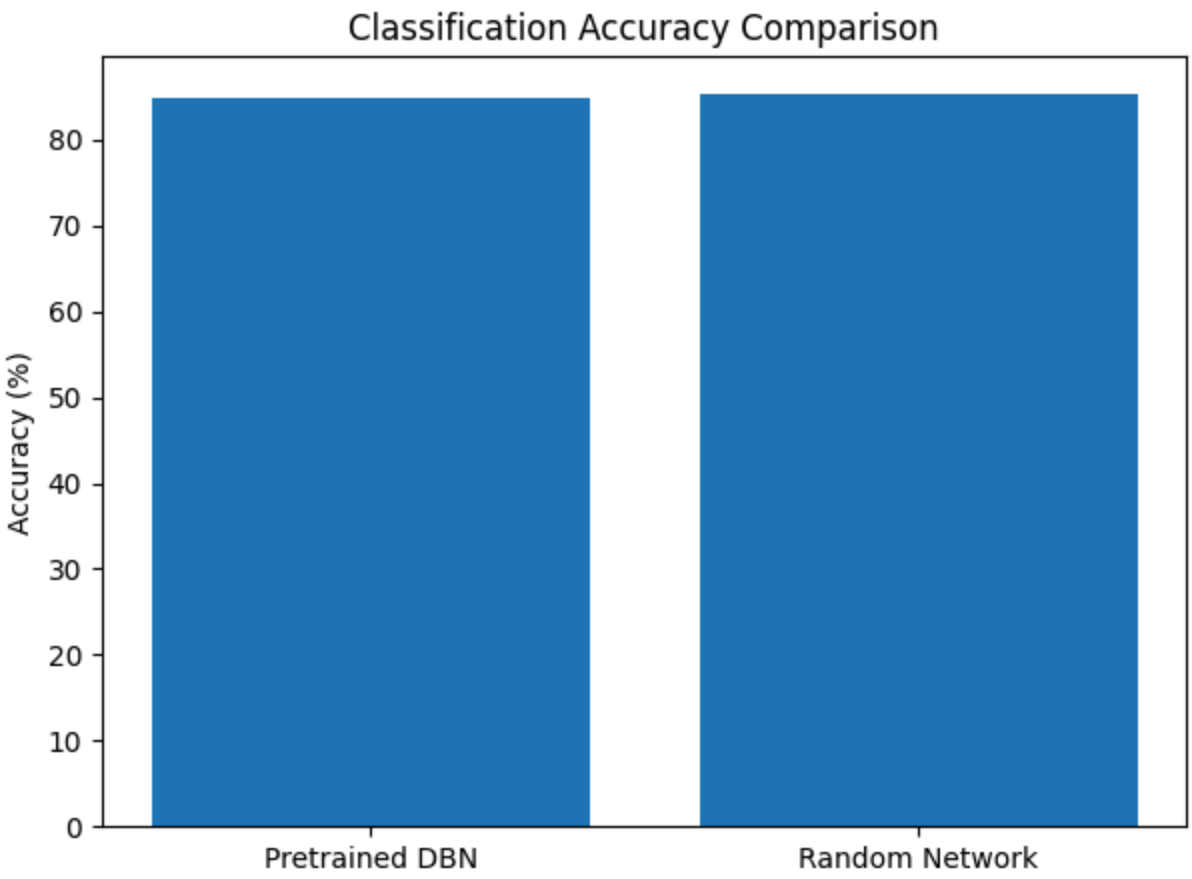
Epoch 10/10

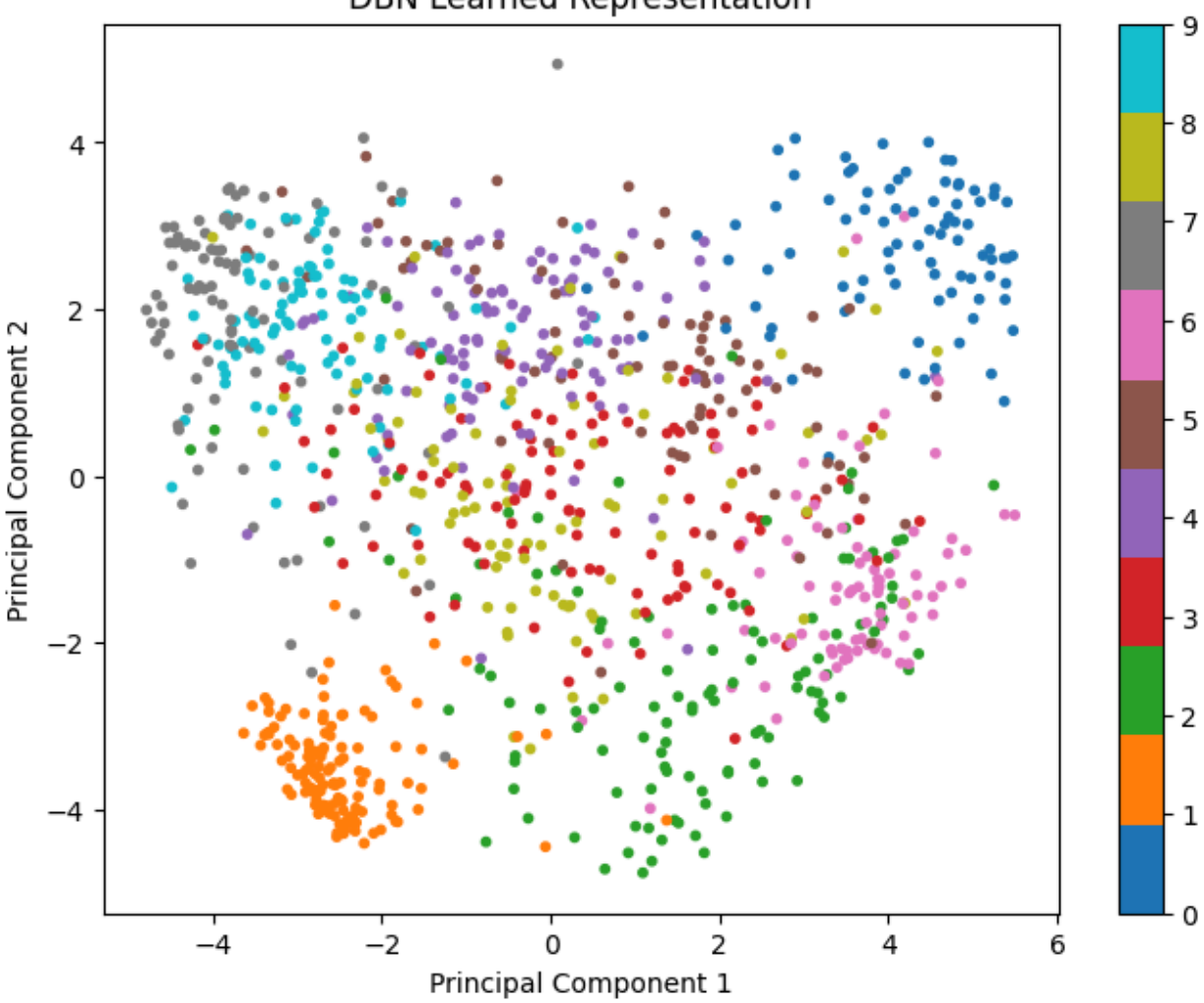
29/29 ━━━━━━━━━━━━━━━━━━━━ **0s** 5ms/step - accuracy: 0.9500 - loss: 0.2317 - val_accuracy: 0.8800 - val_loss: 0.5030

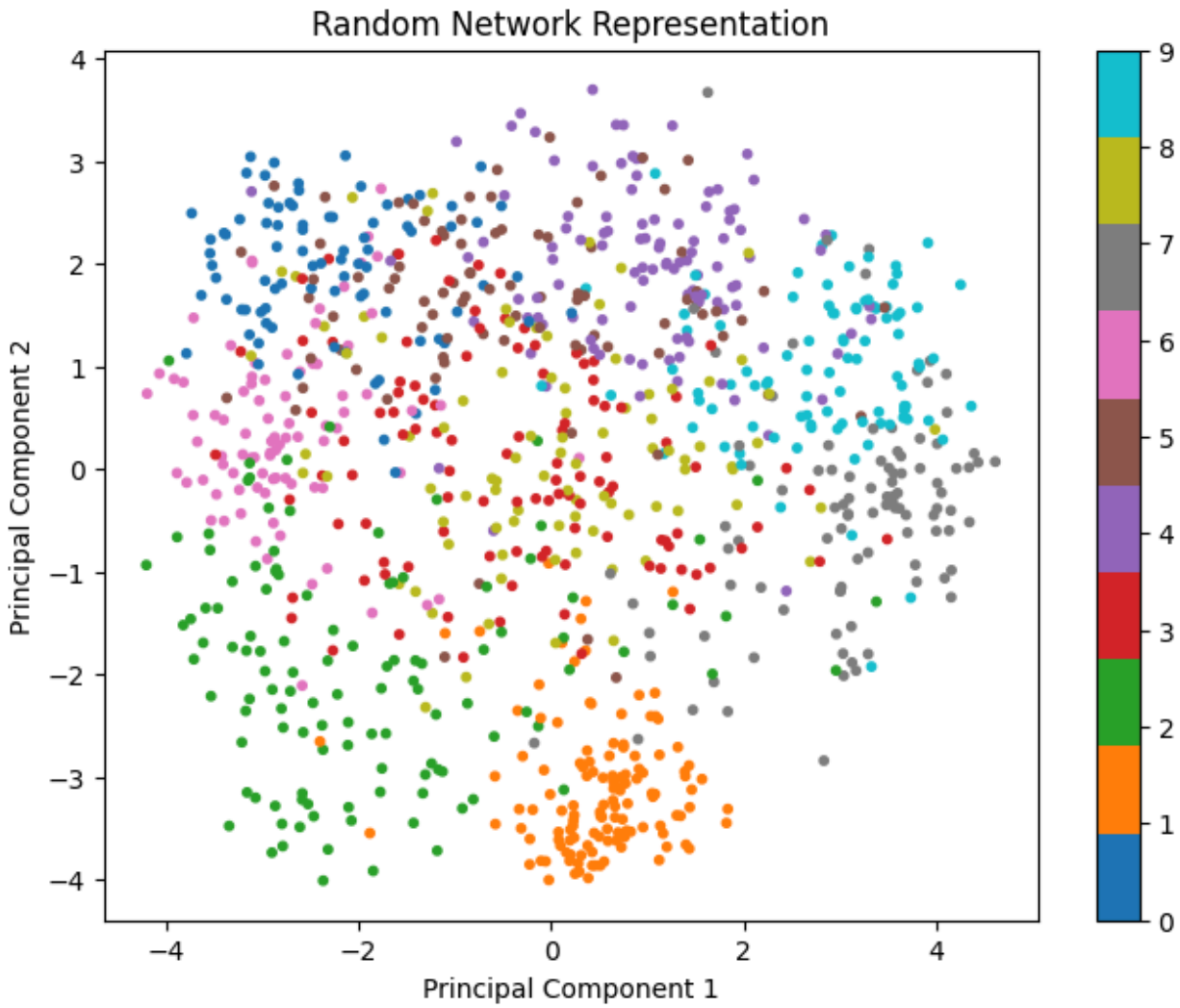
FINAL ACCURACY

Pretrained DBN Accuracy: 84.96 %

Random Network Accuracy: 85.4 %


Classification Accuracy Comparison
Accuracy (%)
0
10
20
30
40
50
60
70
80
Pretrained DBN
Random Network


DBN Learned Representation
Principal Component 2
Principal Component 1
4
2
0
−2
−4
−4
−2
0
2
4
6
9
8
7
6
5
4
3
2
1
0


Random Network Representation
Principal Component 2
Principal Component 1
4
3
2
1
0
−1
−2
−3
−4
−4
−2
0
2
4
9
8
7
6
5
4
3
2
1
0